

Lab Report: An Agentic AI System for Information Retrieval

Mark Lopes(9913)

Jonathan Gomes(9900)

Abstract

This report details the implementation and evaluation of a simple Agentic AI system designed for information retrieval (IR). Traditional IR and Retrieval-Augmented Generation (RAG) systems follow a fixed pipeline, which can fail if initial retrieval steps yield irrelevant information. This project introduces an agent that autonomously plans, acts using multiple tools, and reflects on its progress to answer user queries. The agent was iteratively improved, first by adding a reflection rule to self-correct based on output quality, and second by enhancing its planning logic to prevent repetitive failures. The results demonstrate that even a simple agentic framework provides significantly more robustness and adaptability than static pipelines, though its performance is ultimately constrained by the capabilities of its individual tools.

1. Introduction

Information Retrieval (IR) systems are fundamental to accessing vast digital libraries. While effective, traditional methods often lack flexibility. The advent of Large Language Models (LLMs) led to Retrieval-Augmented Generation (RAG), which improved answer quality by grounding LLMs in factual data. However, RAG still operates as a rigid, linear pipeline: retrieve, then generate. If the initial retrieval is poor, the entire process fails.

Agentic AI represents the next evolution. An AI agent is an autonomous system that perceives its environment, makes decisions (plans), and takes actions (uses tools) to achieve a goal. In the context of IR, an agent can dynamically choose which information source to query, synthesize findings, and even reflect on its own progress to decide if another search is necessary.

The objective of this lab was to build such an agent. The system was designed with three core components:

1. **Planning:** A simple rule-based engine to decide the next action.
2. **Acting:** A set of tools including a local file search (FAISS), an online search (Wikipedia), and an LLM for synthesis (FLAN-T5).
3. **Reflection:** A mechanism for the agent to evaluate its own output and decide whether to continue its work.

This report documents the implementation steps, from the baseline agent to a more robust version, and analyzes its behavior across various queries.

2. Implementation Steps

The implementation followed an iterative process of building a baseline agent and then improving it based on observed failures. The environment was a Google Colab notebook.

Step 1: Baseline Agent Setup

The initial provided template defined the core architecture. This included three tools: LocalCorpusTool for searching local text files, WikipediaTool for online queries, and LLMTTool for generating answers. The AgentICRAgent class contained the main plan, act, and reflect methods. The initial logic was very simple: try a local search first, and if evidence is found, synthesize an answer immediately.

Step 2: Corpus Expansion and Initial Testing

To test the local retrieval capability, the sample_corpus directory was populated with three files: rag_explained.txt, llms_intro.txt, and a custom-added solar_power.txt. Initial tests showed the agent could retrieve these files when the query was relevant.

Step 3: First Modification - Enhancing Reflection

A key failure was observed when a query had no relevant local documents. The agent would retrieve irrelevant files and generate a nonsensical, very short answer (e.g., [2].). To address this, the reflect() method was modified. A rule was added to check if a generated final_answer was too short (fewer than 50 words). If so, the agent would consider the answer a failure, discard it (state.final_answer = None), and continue its execution loop, allowing it to try another tool.

Key addition to the reflect() method

```
if state.final_answer and len(state.final_answer.split()) < 50:
```

```
    print("--- Reflection: The generated answer is too short...")
```

```
    state.final_answer = None # Erase the bad answer
```

```
    # ... logic to ensure it can continue ...
```

```
    return True
```

Step 4: Second Modification - Improving the Plan

The reflection enhancement introduced a new problem: an infinite loop. The agent would synthesize a short answer, reflect and discard it, but its simple plan() logic would see that evidence *still existed* and immediately try to synthesize again with the same bad evidence. To fix this, the plan() method was made more intelligent. It was updated to be aware of which tools had already been used. If it had only used the local tool, its next plan would be to try Wikipedia, rather than re-synthesizing fruitlessly.

More intelligent logic in the plan() method

```
used_sources = {e.get("source", "").split(':')[0] for e in state.evidence}
```

```
if 'local' in used_sources and 'wikipedia' not in used_sources:
```

```
    return "TRY_WIKIPEDIA"
```

This change broke the loop and enabled true multi-tool sequential use, completing the agent's core logic.

3. Results and Discussion

Result 1: Baseline Agent Failure

When the baseline agent was queried about a topic not in the local corpus, it failed predictably. It retrieved irrelevant documents and the LLM synthesizer could not produce a coherent answer.

- **Observation:** This demonstrates the brittleness of a static pipeline. The initial retrieval failed, and the system had no mechanism to recover. The agent prematurely proceeded to synthesis with low-quality evidence.

Result 2: The Corrected Agentic Loop in Action

After implementing the improved `reflect()` and `plan()` methods, the same query was run again. This time, the agent demonstrated a complete, successful reasoning loop.

```
print("Completed in {}".format(datetime.now() - start_time))
print("\n" * 80)

Device set to use cpu
Token indices sequence length is longer than the specified maximum sequence length for this model (878 > 512)
--- Reflection: The generated answer is too short. The agent will try to gather more evidence. ---
=====
Query: what is genshin impact

- Steps -
{'action': 'TRY_LOCAL', 'num_docs': 3}
{'action': 'TRY_WIKIPEDIA', 'num_docs': 2}
{'action': 'SYNTHESIZE', 'used_docs': 5}
{'action': 'SYNTHESIZE', 'used_docs': 5}

- Evidence -
[1] local:rag_explained.txt
[2] local:llms_intro.txt
[3] local:solar_power.txt
[4] wikipedia:Paimon (Genshin Impact) (https://en.wikipedia.org/wiki/Paimon\_\(Genshin\_Impact\)\))
[5] wikipedia:Furina (Genshin Impact) (https://en.wikipedia.org/wiki/Furina\_\(Genshin\_Impact\)\))

- Final Answer -
a 2020 action role-playing gacha game
-----
Completed in 28.45s
=====
```

- **Observation:** The agent's steps clearly show the desired behavior:
 1. It first tried a `TRY_LOCAL` action and retrieved irrelevant files.
 2. It attempted to `SYNTHESIZE`, producing a short, invalid answer.
 3. The `Reflection`: message shows our new rule kicking in, rejecting the answer.
 4. The improved `plan()` method then correctly chose `TRY_WIKIPEDIA` as the next action, gathering new, relevant evidence.
 5. Finally, it synthesized a coherent answer from the combined evidence.

Result 3: Limitation of the Synthesis Tool

Even with perfect agentic logic, the system is constrained by its tools. When queried about "Impact of climate change on agriculture in India", the agent correctly gathered all relevant evidence. However, the FLAN-T5 model, overwhelmed by a long and noisy prompt (containing both relevant and irrelevant info), failed to generate a summary and instead outputted only the citation of the most relevant source.

```

Device set to use cpu
Token indices sequence length is longer than the specified maximum sequence length for this model (901 > 512). Running this sequen
--- Reflection: The generated answer is too short. The agent will try to gather more evidence. ---
=====
Query: Impact of climate change on agriculture in India

- Steps -
{'action': 'TRY_LOCAL', 'num_docs': 3}
{'action': 'TRY_WIKIPEDIA', 'num_docs': 3}
{'action': 'SYNTHESIZE', 'used_docs': 6}
{'action': 'SYNTHESIZE', 'used_docs': 6}

- Evidence -
[1] local:solar_power.txt
[2] local:llms_intro.txt
[3] local:rag_explained.txt
[4] wikipedia:Effects of climate change on agriculture (https://en.wikipedia.org/wiki/Effects\_of\_climate\_change\_on\_agriculture)
[5] wikipedia:Climate change (https://en.wikipedia.org/wiki/Climate\_change)
[6] wikipedia:Climate change in India (https://en.wikipedia.org/wiki/Climate\_change\_in\_India)

- Final Answer -
[4]
-----
Completed in 20.36s
=====

```

- **Observation:** This is a crucial finding. The agent's reasoning loop (plan, act, reflect) was flawless. However, the LLMTTool itself failed at the final step. This highlights that an agent is a complex system where failure can occur in any component, not just the high-level logic.

4. Reflection Answers

1. How does the agent plan, act, and reflect?

- **Plan:** The agent plans via a rule-based plan() method that inspects the current state (e.g., what evidence has been gathered, which tools have been used) to decide the next action string (e.g., TRY_LOCAL, SYNTHESIZE).
- **Act:** The agent acts via the act() method, which takes the action string from the planner and executes it by calling the appropriate tool (LocalCorpusTool, WikipediaTool, or LLMTTool).
- **Reflect:** The agent reflects via the reflect() method, which evaluates the outcome of an action. In our case, it checked if a final answer was generated and if that answer met a quality standard (word count > 50). This determines if the agent's goal is met or if it needs to continue working.

2. How does this agentic behavior differ from static IR/RAG pipelines?

The key difference is **dynamism and recovery**. A static RAG pipeline follows a fixed retrieve -> generate path. If retrieval fails, the entire system fails. Our agent demonstrated the ability to recover from a failed retrieval. It acted, reflected on the poor result, and re-planned to use a different tool. This iterative, self-correcting loop is the hallmark of agentic behavior and is absent in static systems.

3. What are the limitations of this agent?

- **Simple Planning:** The planning is purely rule-based and not truly intelligent. It cannot, for example, decompose a complex question into sub-questions.
- **Tool Fallibility:** As seen in Result 3, the agent is only as good as its weakest tool. The synthesis LLM proved to be a bottleneck, failing on complex prompts.

- **Lack of Deeper Understanding:** The agent does not understand the *content* of the evidence. It only knows whether it has evidence and where it came from. A more advanced agent might score the relevance of evidence before deciding to synthesize.

5. Conclusion

This lab successfully demonstrated the core principles of Agentic AI for information retrieval. By implementing an agent with the ability to plan, use multiple tools, and reflect on its own outputs, we created a system significantly more robust than a static RAG pipeline. The iterative process of identifying a failure (e.g., premature synthesis) and encoding a solution in the agent's logic (e.g., the reflection rule) highlights the unique development paradigm of agent-based systems. While the final agent was still limited by its simple planner and the capabilities of its underlying LLM tool, it served as a powerful proof-of-concept for the flexibility and resilience offered by the agentic approach.

Appendix: Final Python Code

```
# Install required packages quietly
```

```
!pip install wikipedia sentence-transformers faiss-cpu transformers torch --upgrade -q
```

```
# Create the sample corpus directory
```

```
!mkdir -p sample_corpus
```

```
%%writefile sample_corpus/rag_explained.txt
```

Retrieval-Augmented Generation (RAG) is an AI framework that improves the quality of language model responses by grounding them in external sources of knowledge.

Instead of relying solely on its pre-trained data, a RAG system first retrieves relevant documents or passages from a knowledge base

(like a collection of text files or a database) and then uses this information as context to generate a more accurate and factual answer.

```
%%writefile sample_corpus/llms_intro.txt
```

A Large Language Model (LLM) is a type of artificial intelligence model trained on vast amounts of text data to understand and generate human-like language.

Key examples include models from the GPT family, LLaMA, and PaLM. They can perform a wide range of tasks, such as translation, summarization, and question-answering,

by predicting the next word in a sequence.

```
# Add this in a new cell and run it
```

```
%%writefile sample_corpus/solar_power.txt
```

Solar power is the conversion of energy from sunlight into electricity. It is a renewable energy source, and its technologies are broadly characterized

as either passive solar or active solar depending on how they capture and distribute solar energy or convert it into solar power.

Active solar techniques include the use of photovoltaic systems, concentrated solar power, and solar water heating to harness the energy.

```
import os
```

```
import json
```

```
import argparse
```

```
import time
```

```
from dataclasses import dataclass, field
```

```
from typing import List, Dict, Any, Optional
```

```
# ---- Tool 0: Utilities
```

```
def truncate(text: str, n: int = 1200) -> str:
```

```
    return text if len(text) <= n else text[:n] + "...[truncated]"
```

```
# ---- Tool 1: Local Corpus Retrieval (FAISS + sentence-transformers)
```

```
class LocalCorpusTool:
```

```
    def __init__(self, docs_dir: str, index_path: str, model_name: str = "sentence-transformers/all-MiniLM-L6-v2"):
```

```
        self.docs_dir = docs_dir
```

```
        self.index_path = index_path
```

```
        self.model_name = model_name
```

```
        self.emb = None
```

```
        self.faiss = None
```

```
        self.doc_texts = []
```

```
        self.doc_names = []
```

```
    if not os.path.isdir(docs_dir):
```

```
        os.makedirs(docs_dir, exist_ok=True)
```

```
    def _lazy_import(self):
```

```
        global faiss, SentenceTransformer
```

```
        import faiss # type: ignore
```

```
        from sentence_transformers import SentenceTransformer # type: ignore
```

```
        self.faiss = faiss
```

```
        if self.emb is None:
```

```
            self.emb = SentenceTransformer(self.model_name)
```

```
    def _load_docs(self):
```

```
        self.doc_texts, self.doc_names = [], []
```

```
        for fn in os.listdir(self.docs_dir):
```

```
            if fn.lower().endswith((".txt", ".md")):
```

```

path = os.path.join(self.docs_dir, fn)

with open(path, "r", encoding="utf-8") as f:

    txt = f.read().strip()

self.doc_texts.append(txt)

self.doc_names.append(fn)

```

```

def build_or_load(self):

    self._lazy_import()

    self._load_docs()

    if not self.doc_texts:

        print("Warning: No documents found in the local corpus directory.")

        self.index = None

        return

    vectors = self.emb.encode(self.doc_texts, show_progress_bar=False)

    d = vectors.shape[1]

    index = self.faiss.IndexFlatIP(d) # cosine via normalized dot

    import numpy as np

    norms = np.linalg.norm(vectors, axis=1, keepdims=True) + 1e-10

    vectors = vectors / norms

    index.add(vectors.astype("float32"))

    self.index = index

```

```

def search(self, query: str, k: int = 3) -> List[Dict[str, str]]:

    self._lazy_import()

    if not hasattr(self, "index"):

        self.build_or_load()

    if self.index is None:

        return []

    qv = self.emb.encode([query], show_progress_bar=False)

    import numpy as np

    qv = qv / (np.linalg.norm(qv, axis=1, keepdims=True) + 1e-10)

```



```

D, I = self.index.search(qv.astype("float32"), k)

results = []

for idx in I[0]:
    if idx < 0 or idx >= len(self.doc_texts):
        continue

    results.append({"source": f"local:{self.doc_names[idx]}", "text": self.doc_texts[idx]})

return results

```

---- Tool 2: Wikipedia Search (online)

```
class WikipediaTool:
```

```

    def __init__(self, lang: str = "en", max_chars: int = 1200):
        self.lang = lang
        self.max_chars = max_chars

```

```

    def _lazy_import(self):
        global wikipedia
        import wikipedia # type: ignore
        wikipedia.set_lang(self.lang)

```

```

    def search_and_summarize(self, query: str, n: int = 3) -> List[Dict[str, str]]:

```

```

        try:
            self._lazy_import()

            titles = wikipedia.search(query)[:n]

            results = []

            for t in titles:
                try:
                    page = wikipedia.page(t, auto_suggest=False)
                    summ = wikipedia.summary(t, sentences=6)
                    results.append({
                        "source": f"wikipedia:{page.title}",
                        "url": page.url,

```

```

        "text": summ[: self.max_chars]
    })

except Exception:

    continue

return results

except Exception:

    return []

```

---- Tool 3: LLM for synthesis (FLAN-T5 on CPU)

class LLMTool:

```

def __init__(self, model_name: str = "google/flan-t5-base", max_new_tokens: int = 256):
    self.model_name = model_name
    self.max_new_tokens = max_new_tokens
    self.pipe = None

```

```

def _lazy_import(self):

```

```

    if self.pipe is None:
        from transformers import pipeline # type: ignore
        self.pipe = pipeline("text2text-generation", model=self.model_name)

```

```

def answer(self, query: str, evidence: List[Dict[str, str]]) -> str:

```

```

    self._lazy_import()
    chunks = []
    for i, ev in enumerate(evidence, 1):
        src = ev.get("source", "unknown")
        txt = ev.get("text", "")
        chunks.append(f"[{i}] {src} :: {txt}")

```

```

    context = "\n".join(chunks)

```

```

    prompt = (

```

```

        "You are an IR agent. Synthesize a concise answer (120-200 words) using the evidence below. "

```

```

        "Cite sources in-line like [1], [2]. If evidence is insufficient, say what is missing.\n\n"
    )

```

```

        f"Question: {query}\n\nEvidence:\n{context}\n\nAnswer:"
    )

    out = self.pipe(prompt, max_new_tokens=self.max_new_tokens)[0]["generated_text"]
    return out.strip()

# ---- Agent

@dataclass
class AgentConfig:

    max_steps: int = 3

    use_wikipedia: bool = True

    use_local: bool = True

@dataclass
class AgentState:

    query: str

    steps: List[Dict[str, Any]] = field(default_factory=list)

    evidence: List[Dict[str, str]] = field(default_factory=list)

    final_answer: Optional[str] = None

class AgenticIRAgent:

    def __init__(self, docs_dir: str, memory_path: str):

        self.local_tool = LocalCorpusTool(docs_dir, index_path=os.path.join(docs_dir, "faiss.index"))

        self.wiki_tool = WikipediaTool()

        self.llm_tool = LLMTool()

        self.memory_path = memory_path

        self.memory = self._load_memory()

    def _load_memory(self) -> Dict[str, Any]:

        if os.path.exists(self.memory_path):

            try:

                return json.load(open(self.memory_path, "r", encoding="utf-8"))

```

```

        except Exception:
            return {}
    return {}

def _save_memory(self):
    try:
        json.dump(self.memory, open(self.memory_path, "w", encoding="utf-8"), indent=2,
ensure_ascii=False)
    except Exception:
        pass

# This is the new, smarter plan method
def plan(self, state: AgentState) -> str:
    # Check which sources have been used based on the evidence
    used_sources = {e.get("source", "").split(':')[0] for e in state.evidence}

    # Priority 1: If we have evidence from multiple sources, it's time to synthesize.
    if len(used_sources) > 1 and state.evidence:
        return "SYNTHESIZE"

    # Priority 2: If we only have local evidence, the next logical step is to try Wikipedia.
    if 'local' in used_sources and 'wikipedia' not in used_sources:
        return "TRY_WIKIPEDIA"

    # Priority 3: If we only have Wikipedia evidence, try the local corpus next.
    if 'wikipedia' in used_sources and 'local' not in used_sources:
        return "TRY_LOCAL"

    # Priority 4: If we have no evidence at all, start with a tool. Default to local.
    if not state.evidence:
        return "TRY_LOCAL"

```

```
# Default Fallback: If none of the above, synthesize with what we have.
```

```
return "SYNTHESIZE"
```

```
def act(self, action: str, state: AgentState) -> AgentState:
```

```
    if action == "TRY_LOCAL":
```

```
        docs = self.local_tool.search(state.query, k=3)
```

```
        state.steps.append({"action": action, "num_docs": len(docs)})
```

```
        state.evidence.extend(docs)
```

```
        self.memory["last_sources"] = "local"
```

```
    elif action == "TRY_WIKIPEDIA":
```

```
        docs = self.wiki_tool.search_and_summarize(state.query, n=3)
```

```
        state.steps.append({"action": action, "num_docs": len(docs)})
```

```
        state.evidence.extend(docs)
```

```
        self.memory["last_sources"] = "wikipedia"
```

```
    elif action == "SYNTHESIZE":
```

```
        answer = self.llm_tool.answer(state.query, state.evidence)
```

```
        state.steps.append({"action": action, "used_docs": len(state.evidence)})
```

```
        state.final_answer = answer
```

```
    else:
```

```
        state.steps.append({"action": "NOOP"})
```

```
    return state
```

```
# This is the new code for Cell 5
```

```
def reflect(self, state: AgentState) -> bool:
```

```
    """Return True if we should continue iterating."""
```

```
    # NEW RULE: If the answer is too short (<50 words), it's not good enough. Keep trying.
```

```
    if state.final_answer and len(state.final_answer.split()) < 50:
```

```
        print("--- Reflection: The generated answer is too short. The agent will try to gather more  
evidence. ---")
```

```
        state.final_answer = None # Erase the bad answer
```

```

# Check if there are other tools to try
sources = {e.get("source", "").split(':')[0] for e in state.evidence}
if 'local' in sources and 'wikipedia' in sources:
    return False # We've already used all tools, so stop.

# If we haven't used all tools, continue running.
return True

# Original stopping condition: if we have a final answer (that passed the length check), stop.
if state.final_answer:
    return False

# Original continuation conditions
if not state.evidence:
    return True

sources = [e.get("source", "") for e in state.evidence]
has_local = any(s.startswith("local:") for s in sources)
has_wiki = any(s.startswith("wikipedia:") for s in sources)
if not (has_local and has_wiki) and len(state.steps) < 2:
    return True

return True

def run(self, query: str, cfg: AgentConfig) -> AgentState:
    state = AgentState(query=query)
    for step in range(cfg.max_steps):
        action = self.plan(state)
        state = self.act(action, state)
        if not self.reflect(state):

```

```

        break

    if state.final_answer is None and state.evidence:
        state = self.act("SYNTHESIZE", state)
    self._save_memory()
    return state

print("Agent and Tool classes defined successfully.")

QUERY = "Impact of climate change on agriculture in India"

DOCS_DIR = "sample_corpus"
MEMORY_PATH = "memory.json"
MAX_STEPS = 3
# --- End of Configuration ---

# Initialize and run the agent
agent = AgentICRAgent(docs_dir=DOCS_DIR, memory_path=MEMORY_PATH)
cfg = AgentConfig(max_steps=MAX_STEPS)
t0 = time.time()
state = agent.run(QUERY, cfg)
dt = time.time() - t0

# Print the results
print("="*80)
print(f"Query: {QUERY}")
print("\n- Steps -")
for s in state.steps:
    print(s)
print("\n- Evidence -")
for i, ev in enumerate(state.evidence, 1):
    src = ev.get("source", "")

```

```

url = ev.get("url", "")

print(f"[{i}] {src} {'(' + url + ') ' if url else ''}")

print("\n- Final Answer -")

print(state.final_answer or "(No answer generated)")

print("-"*80)

print(f"Completed in {dt:.2f}s")

print("="*80)

QUERY = "What is solar power"


DOCS_DIR = "sample_corpus"

MEMORY_PATH = "memory.json"

MAX_STEPS = 3

# --- End of Configuration ---


# Initialize and run the agent
agent = AgenticIRAgent(docs_dir=DOCS_DIR, memory_path=MEMORY_PATH)

cfg = AgentConfig(max_steps=MAX_STEPS)

t0 = time.time()

state = agent.run(QUERY, cfg)

dt = time.time() - t0


# Print the results

print("="*80)

print(f"Query: {QUERY}")

print("\n- Steps -")

for s in state.steps:

    print(s)

print("\n- Evidence -")

for i, ev in enumerate(state.evidence, 1):

    src = ev.get("source", "")

    url = ev.get("url", "")

```



```

        print(f"[{i}] {src} {'('+'url+')' if url else ''}")
print("\n- Final Answer -")
print(state.final_answer or "(No answer generated)")
print("-"*80)
print(f"Completed in {dt:.2f}s")
print("-"*80)
QUERY = "what is genshin impact"

DOCS_DIR = "sample_corpus"
MEMORY_PATH = "memory.json"
MAX_STEPS = 3
# --- End of Configuration ---

# Initialize and run the agent
agent = AgenticIRAgent(docs_dir=DOCS_DIR, memory_path=MEMORY_PATH)
cfg = AgentConfig(max_steps=MAX_STEPS)
t0 = time.time()
state = agent.run(QUERY, cfg)
dt = time.time() - t0

# Print the results
print("-"*80)
print(f"Query: {QUERY}")
print("\n- Steps -")
for s in state.steps:
    print(s)
print("\n- Evidence -")
for i, ev in enumerate(state.evidence, 1):
    src = ev.get("source", "")
    url = ev.get("url", "")
    print(f"[{i}] {src} {'('+'url+')' if url else ''}")

```

```
print("\n- Final Answer -")
print(state.final_answer or "(No answer generated)")
print("-"*80)
print(f"Completed in {dt:.2f}s")
print("="*80)
```